

Exercice 1 :

1. Fichier Etudiant.h

```
#pragma once // (On peut utiliser #pragma once au lieu de #ifndef et #define et #endif)
#include <string>
using namespace std;

class Etudiant {
private:
    static int compteur; // Pour auto-incrémentation
    int matricule;
    string nom;
    int nbrNotes;
    float* tabNotes;

public:
    // Constructeur par défaut
    Etudiant();

    // Constructeur avec arguments
    Etudiant(string nom, int nbrNotes);

    // Constructeur de copie
    Etudiant(const Etudiant& e);

    // Destructeur
    ~Etudiant();

    // Getters
    int getMatricule() const;
    string getNom() const;
    int getNbrNotes() const;
    float* getTabNotes() const;

    // Setters
    void setNom(string nom);
    void setNbrNotes(int n);

    // Méthodes
    void saisie();
    void affichage() const;
    float moyenne() const;
    bool admis() const;

    // Méthode statique
    static bool comparer(const Etudiant& e1, const Etudiant& e2);
};
```

2. Fichier Etudiant.cpp

```
#include "Etudiant.h"
#include <iostream>
using namespace std;

// Initialisation du compteur
int Etudiant::compteur = 1;

// Constructeur d'initialisation
Etudiant::Etudiant() {
```

```

    matricule = compteur++;
    nom = "";
    nbrNotes = 0;
    tabNotes = nullptr;
}

// Constructeur avec arguments
Etudiant::Etudiant(string nom, int nbrNotes) {
    matricule = compteur++;
    this->nom = nom;
    this->nbrNotes = nbrNotes;
    tabNotes = new float[nbrNotes];
}

// Constructeur de copie
Etudiant::Etudiant(const Etudiant& e) {
    matricule = e.matricule;
    nom = e.nom;
    nbrNotes = e.nbrNotes;

    // Copie profonde
    if (nbrNotes > 0) {
        tabNotes = new float[nbrNotes];
        for (int i = 0; i < nbrNotes; i++)
            tabNotes[i] = e.tabNotes[i];
    }
    else {
        tabNotes = nullptr;
    }
}

// Destructeur
Etudiant::~Etudiant() {
    delete[] tabNotes;
}

// Getters
int Etudiant::getMatricule() const { return matricule; }
string Etudiant::getNom() const { return nom; }
int Etudiant::getNbrNotes() const { return nbrNotes; }
float* Etudiant::getTabNotes() const { return tabNotes; }

// Setters
void Etudiant::setNom(string nom) { this->nom = nom; }

void Etudiant::setNbrNotes(int n) {
    // Réallocation
    delete[] tabNotes;
    nbrNotes = n;
    tabNotes = new float[n];
}

// Saisie des notes
void Etudiant::saisie() {
    cout << "Nom de l'etudiant : ";
    std::cin >> nom;

    cout << "Nombre de notes : ";
    std::cin >> nbrNotes;

    // allouer le tableau dynamiquement
    delete[] tabNotes;
    tabNotes = new float[nbrNotes];
}

```

```

    // saisir les notes
    for (int i = 0; i < nbrNotes; i++) {
        cout << "Note " << i + 1 << " : ";
        cin >> tabNotes[i];
    }
}

// Affichage
void Etudiant::affichage() const {
    cout << "\nMatricule : " << matricule << endl;
    cout << "Nom : " << nom << endl;
    cout << "Nombre de notes : " << nbrNotes << endl;
    cout << "Notes : ";
    for (int i = 0; i < nbrNotes; i++)
        cout << tabNotes[i] << " ";
    cout << endl;
    cout << "Moyenne : " << moyenne() << endl;
    cout << "Admis : " << (admis() ? "Oui" : "Non") << endl;
}

// Calcul moyenne
float Etudiant::moyenne() const {
    float s = 0;
    for (int i = 0; i < nbrNotes; i++)
        s += tabNotes[i];
    return nbrNotes == 0 ? 0 : s / nbrNotes;
}

// Admis ?
bool Etudiant::admis() const {
    return moyenne() >= 10;
}

// Comparer deux étudiants
bool Etudiant::comparer(const Etudiant& e1, const Etudiant& e2) {
    return e1.moyenne() == e2.moyenne();
}

```

3. Exemple main.cpp

```

#include <iostream>
#include "Etudiant.h"
using namespace std;

int main() {
    cout << "=== Gestion des notes des étudiants ===" << endl;

    // Création de deux étudiants
    Etudiant e1;
    Etudiant e2;

    // Saisie pour le premier étudiant
    cout << "\n--- Saisie du premier étudiant ---" << endl;
    e1.saisie();

    // Saisie pour le deuxième étudiant
    cout << "\n--- Saisie du deuxième étudiant ---" << endl;
    e2.saisie();

    // Affichage des informations
    cout << "\n=== Informations des étudiants ===" << endl;
}

```

```
e1.affichage();
e2.affichage();

// Vérification admis/non admis
cout << "\n=== Statut des étudiants ===" << endl;
cout << "Etudiant 1 : " << (e1.admis() ? "Admis" : "Non admis") << endl;
cout << "Etudiant 2 : " << (e2.admis() ? "Admis" : "Non admis") << endl;

// Comparaison des moyennes
cout << "\n=== Comparaison des moyennes ===" << endl;
if (Etudiant::comparer(e1, e2))
    cout << "Les deux étudiants ont la même moyenne." << endl;
else
    cout << "Les deux étudiants n'ont pas la même moyenne." << endl;

return 0;
}
```